

flooded

```
const int INF = 1e9;
void solve()
{
    int n, m;
    cin >> n >> m;
    // ফ্লয়েড এর জন্য adjacency matrix
    vector<vector<int>> d(n, vector<int>(n, INF));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            int x;
            cin >> x;
            if (i == j)
                d[i][j] = 0; // নিজে থেকে নিজে 0
            else if (x != -1)
                d[i][j] = x; // -1 মানে edge নাই
        }
    }
    // ফ্লয়েড ওয়ারশাল
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    // নেগেটিভ সাইকেল চেক
    for (int i = 0; i < n; i++)
    {
        if (d[i][i] < 0)
        {
            cout << "Negative cycle ache\n";
            return;
        }
    }
    int q;
    cin >> q;
    while (q--)
    {
        int a, b;
        cin >> a >> b;
        cout << "dis(" << a << " >" << b << ") = ";
        if (d[a][b] == INF)
            cout << "INF\n";
        else
            cout << d[a][b] << "\n";
    }
}
```

KRUSKAL

```
const int MAXN = 1e5 + 5;
int parent[MAXN], sz[MAXN];
int find_set(int v) { return v == parent[v] ? v : parent[v] = find_set(parent[v]); }
bool union_sets(int a, int b)
{
    a = find_set(a);
    b = find_set(b);
    if (a == b)
        return false;
    if (sz[a] < sz[b])
        swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    return true;
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        parent[i] = i, sz[i] = 1;

    vector<tuple<int, int, int>> edges(m);
    for (int i = 0, a, b, w; i < m; i++)
    {
        cin >> a >> b >> w;
        edges[i] = {w, a, b};
    }
    sort(edges.begin(), edges.end());

    int mst_weight = 0;
    vector<pair<int, int>> mst_edges;
    for (auto &[w, a, b] : edges)
        if (union_sets(a, b))
        {
            mst_weight += w;
            mst_edges.push_back({a, b});
        }

    cout << "MST এর মোট ওজন: " << mst_weight << "\nMST এর এজ গুলো:\n";
    for (auto &[a, b] : mst_edges)
        cout << a << " - " << b << "\n";
}
```

prims

```
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> adj(n + 1);
    while (m--)
    {
        int a, b, w;
        cin >> a >> b >> w;
        adj[a].push_back({b, w});
        adj[b].push_back({a, w});
    }
    vector<int> vis(n + 1, 0);
    priority_queue<pair<int, pair<int, int>>>, vector<pair<int, pair<int, int>>>, greater<>> pq;
    pq.push({0, {1, -1}}); // start from node 1
    vector<pair<int, int>> spt;
    int sum = 0;
    while (!pq.empty())
    {
        auto [w, p] = pq.top();
        pq.pop();
        int u = p.first, par = p.second;
        if (vis[u])
            continue;
        vis[u] = 1;
        if (par != -1)
        {
            spt.push_back({par, u});
            sum += w;
        }
        for (auto [v, c] : adj[u])
            if (!vis[v])
                pq.push({c, {v, u}});
    }
    if (spt.size() != n - 1)
    {
        cout << -1 << "\n";
        return;
    }
    cout << sum << "\n";
    for (auto [a, b] : spt)
        cout << a << " " << b << "\n";
}
```

TOPO DFS

```
void champ()
{
    int n, m;
    cin >> n >> m;
    vector<int> indeg(n + 1, 0);
    vector<vector<int>> adj(n + 1);
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indeg[b]++;
    }
    priority_queue<int, vector<int>, greater<>> pq;
    for (int i = 1; i <= n; i++)
        if (indeg[i] == 0)
            pq.push(i);
    vector<int> ans;
    while (!pq.empty())
    {
        int u = pq.top();
        pq.pop();
        ans.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0)
                pq.push(v);
    }
    if (ans.size() != n)
        cout << "Cycle here\n";
    else
        for (int x : ans)
            cout << x << " ";
}
```

```
stack<int> st;
void dfs(vector<vector<int>> &adj, vector<int> &vis, int u)
{
    vis[u] = 1;
    for (int v : adj[u])
        if (!vis[v])
            dfs(adj, vis, v);
    st.push(u);
}
void champ()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> vis(n + 1, 0);
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    for (int i = 1; i <= n; i++)
        if (!vis[i])
            dfs(adj, vis, i);
    while (!st.empty())
    {
        cout << st.top() << " ";
        st.pop();
    }
}
```

T
O
P
O

SSC

```
#include <bits/stdc++.h>
using namespace std;
// প্রথম DFS: টপোলজিকাল অর্ডার স্ট্যাকে রাখে
void dfs1(vector<vector<int>> &adj, int u, vector<int> &vis, stack<int> &st)
{
    vis[u] = 1;
    for (auto v : adj[u])
        if (!vis[v])
            dfs1(adj, v, vis, st);
    st.push(u);
}
// দ্বিতীয় DFS: রিভার্স গ্রাফে চলবে, একেকটা SCC প্রিন্ট করবে
void dfs2(vector<vector<int>> &rev, int u, vector<int> &mark)
{
    mark[u] = 1;
    cout << u << " ";
    for (auto v : rev[u])
        if (!mark[v])
            dfs2(rev, u, mark);
}
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1, rev(n + 1));
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        rev[b].push_back(a); // reverse edge
    }
    // Step 1: DFS করে stack এ order রাখা
    vector<int> vis(n + 1, 0);
    stack<int> st;
    for (int i = 1; i <= n; i++)
        if (!vis[i])
            dfs1(adj, i, vis, st);
    // Step 2: Stack থেকে node বের করে রিভার্স গ্রাফে DFS
    চালানো
    vector<int> mark(n + 1, 0);
    while (!st.empty())
    {
        int u = st.top();
        st.pop();
        if (!mark[u])
        {
            dfs2(rev, u, mark);
            cout << "\n"; // প্রতিটি SCC নতুন লাইনে
        }
    }
}
```

```
#ifndef ONLINE_JUDGE
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);
#endif
```

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long int
#define endl "\n"
#define ot() ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
int main()
{
    #ifndef ONLINE_JUDGE
        freopen("in.txt", "r", stdin);
        freopen("out.txt", "w", stdout);
    #endif
    ot();
}
```

BUBble

```
void bubbleSort(vector<int> &arr)
{
    int n = arr.size();
    for (int i = 0; i < n - 1; i++)
    {
        // প্রতিটি pass এ largest element তার correct place
        এ চলে যায়
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

QUICK SORT

```
int partition(vector<int> &a, int l, int r)
{
    int p = a[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (a[j] <= p)
            swap(a[++i], a[j]);
    swap(a[i + 1], a[r]);
    return i + 1;
}
void quickSort(vector<int> &a, int l, int r)
{
    if (l < r)
    {
        int pi = partition(a, l, r);
        quickSort(a, l, pi - 1);
        quickSort(a, pi + 1, r);
    }
}
int main()
{
    vector<int> a = {10, 7, 8, 9, 1, 5};
    quickSort(a, 0, a.size() - 1);
    for (int x : a)
        cout << x << " ";
```

Insertion sort

```
int partition(vector<int> &a, int l, int r)
{
    int p = a[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (a[j] <= p)
            swap(a[++i], a[j]);
    swap(a[i + 1], a[r]);
    return i + 1;
}
void quickSort(vector<int> &a, int l, int r)
{
    if (l < r)
    {
        int pi = partition(a, l, r);
        quickSort(a, l, pi - 1);
        quickSort(a, pi + 1, r);
    }
}
int main()
{
    vector<int> a = {10, 7, 8, 9, 1, 5};
    quickSort(a, 0, a.size() - 1);
    for (int x : a)
        cout << x << " ";
```

MERGE

```
// Merge function: দুইটি sorted half merge করবে
void merge(vector<int> &arr, int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int i = 0; i < n2; i++)
        R[i] = arr[m + 1 + i];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}
// Merge Sort function
void mergeSort(vector<int> &arr, int l, int r)
{
    if (l >= r)
        return; // base case: single element
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}
int main()
{
    vector<int> arr = {38, 27, 43, 3, 9, 82, 10};
    cout << "Before sorting: ";
    for (int x : arr)
        cout << x << " ";
    cout << "\n";
    mergeSort(arr, 0, arr.size() - 1);
    cout << "After sorting: ";
    for (int x : arr)
        cout << x << " ";
    cout << "\n";
    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;
#define op() \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);
#define test(t) \
    int t; \
    cin >> t; \
    while (t--)
#define MOD 1000000007
#define MOD1 998244353
#define endl "\n"
#define pb push_back
#define mp make_pair
#define ff first
#define ss second
#define all(vec) vec.begin(), vec.end()
#define PI 3.141592653589793238462
#define debug(x) cout << #x << " " << x << endl;
#define ll long long int
```

Bipartite DFS

```
bool dfs(int u, int c, vector<int> &col,
vector<vector<int>> &adj)
{
    col[u] = c;
    for (int v : adj[u])
    {
        if (col[v] == c)
            return false;
        if (col[v] == -1 && !dfs(v, c ^ 1, col, adj))
            return false;
    }
    return true;
}

void champ()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> col(n + 1, -1);
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    for (int i = 1; i <= n; i++)
        if (col[i] == -1 && !dfs(i, 0, col, adj))
        {
            cout << "Not Bipartite\n";
            return;
        }
    cout << "Bipartite\n";
}
```

Bipartite BFS

```
// BFS to check bipartite
bool bfs(int u, vector<vector<int>> &adj, vector<int> &col)
{
    queue<int> q;
    q.push(u);
    col[u] = 0;
    while (!q.empty())
    {
        int cur = q.front();
        q.pop();
        for (int v : adj[cur])
        {
            if (col[v] == col[cur])
                return false;
            if (col[v] == -1)
            {
                col[v] = col[cur] ^ 1;
                q.push(v);
            }
        }
    }
    return true;
}

// DFS to count components
void dfs(int u, vector<int> adj[], vector<int> &vis)
{
    vis[u] = 1;
    for (int v : adj[u])
        if (!vis[v])
            dfs(v, adj, vis);
}

int main()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj1(n + 1);
    vector<int> col(n + 1, -1);
    vector<int> adj2[n + 1], vis(n + 1, 0);
    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        adj1[a].push_back(b);
        adj1[b].push_back(a);
        adj2[a].push_back(b);
        adj2[b].push_back(a);
    }

    // Bipartite check
    for (int i = 1; i <= n; i++)
        if (col[i] == -1 && !bfs(i, adj1, col))
        {
            cout << "Not Bipartite\n";
            break;
        }

    // Count components
    int count = 0;
    for (int i = 1; i <= n; i++)
        if (!vis[i])
        {
            dfs(i, adj2, vis);
            count++;
        }
}
```

Is it cycle?

```
// ----- DFS দিয়ে সাইকেল চেক -----
bool dfs(vector<vector<int>> &adj, vector<int> &vis,
int u, int p)
{
    vis[u] = 1;
    for (auto v : adj[u])
    {
        if (vis[v] && v != p)
            return true;
        if (!vis[v])
        {
            if (dfs(adj, vis, v, u))
                return true;
        }
    }
    return false;
}

// ----- BFS দিয়ে সাইকেল চেক -----
bool bfs(vector<vector<int>> &adj, int n)
{
    vector<int> vis(n + 1, 0);
    queue<pair<int, int>> q;

    for (int s = 1; s <= n; s++)
    {
        if (!vis[s])
        {
            q.push({s, -1});
            vis[s] = 1;

            while (!q.empty())
            {
                auto [u, p] = q.front();
                q.pop();

                for (auto v : adj[u])
                {
                    if (vis[v] && v != p)
                        return true;
                    if (!vis[v])
                    {
                        vis[v] = 1;
                        q.push({v, u});
                    }
                }
            }
        }
    }
    return false;
}

// ----- main function -----
void champ()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    while (m--)
    {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    // ----- DFS check -----
    vector<int> vis(n + 1, 0);
    for (int i = 1; i <= n; i++)
        if (!vis[i])
        {
            if (dfs(adj, vis, i, -1))
            {
                cout << "Cycle (DFS)\n";
                return;
            }
        }

    // ----- BFS check -----
    if (bfs(adj, n))
        cout << "Cycle (BFS)\n";
    else
        cout << "No Cycle (BFS)\n";
}
```

Dijkstra

```
const int INF = 1e9;
void champ()
{
    int n, m;
    cin >> n >> m;
    vector<int> dis(n + 1, INF);
    vector<vector<pair<int, int>>> adj(n + 1);
    while (m--)
    {
        int a, b, w;
        cin >> a >> b >> w;
        adj[a].push_back({b, w});
        adj[b].push_back({a, w});
    }
    int start;
    cin >> start;
    dis[start] = 0;
    priority_queue<pair<int, int>, vector<pair<int,
int>>, greater<pair<int, int>>> pq;
    pq.push({0, start});
    while (!pq.empty())
    {
        auto [w, node] = pq.top();
        pq.pop();
        if (w > dis[node])
            continue;
        for (auto [next, cost] : adj[node])
        {
            if (dis[next] > w + cost)
            {
                dis[next] = w + cost;
                pq.push({dis[next], next});
            }
        }
    }
    // Print shortest distances from start
    for (int i = 1; i <= n; i++)
        cout << dis[i] << " ";
    cout << endl;
}
```

```
// Print shortest distances
from start
for (int i = 1; i <= n; i++)
{
    cout << dis[i] << " ";
}
cout << endl;
```

```
cout << "No Cycle (DFS)\n";
BFS_CHECK:
// ---- BFS check ----
if (bfs(adj, n))
    cout << "Cycle (BFS)\n";
else
    cout << "No Cycle (BFS)\n";
}
```